Python Cheat Sheet

1. Basics

- Python is a high-level, interpreted, general-purpose programming language.
- Easy to learn and read (simple syntax).
- Everything in Python is an object.
- # is used for single line comment
- Use triple quotes """ """ or ''' ''' for multi-line string / docstring.

Hello World

```
print("Hello, World!")
```

Variables

```
x = 10
name = "Aman"
is_valid = True
```

2. Data Types

- int : -10, 0, 25
- float : 3.14, -0.01
- str : "hello"
- bool : True, False
- list : [1, 2, 3]
- tuple : (1, 2, 3)
- set : {1, 2, 3}
- dict : {"name": "Aman", "age": 25}
- NoneType : None

3. Type Conversion

- int() float() str()
- list() tuple() set()
- dict() bool()

Example:

```
x = "10"
int(x)      # 10
float(x)    # 10.0
str(100)    # "100"
```

4. Operators

Arithmetic

+ - * / // % **

Comparison

== != > < >= <=

Logical

and or not

Assignment

= += -= *= /= %= //= **=

Bitwise

& | ^ ~ << >>

Membership

in not in

Identity

is is not

5. Input / Output

Input

```
name = input("Enter name: ")
```

Output

```
print("Hello", name)
print(f"Hello {name}")
```

Formatting

```
age = 25
print("Age: {}".format(age))
print(f"Age: {age}")
```

6. Control Flow

If - Elif - Else

```
if condition:
    pass
elif condition:
    pass
else:
    pass
```

For Loop

```
for i in range(5):
    print(i)

for item in list:
    print(item)
```

While Loop

```
while condition:
    pass
```

```
range(stop)          -> 0 to stop-1
range(start, stop)   -> start to stop-1
range(start, stop, step)
  step can be +ve or -ve
```

7. Strings

Common Operations

- s[ ] # indexing
- s[start : end] # slicing
- len(s) # length
- s.lower() # to lowercase
- s.upper() # to uppercase
- s.strip() # remove spaces
- s.replace(old, new)
- s.split(delimiter)
- 'sub' in s # membership

String Methods

- capitalize()
- title()
- count(sub)
- find(sub)
- index(sub)
- startswith(x)
- endswith(x)
- isalpha()
- isdigit()
- isalnum()
- isspace()
- join(iterable)
- center(width)
- ljust() / rjust()

String Formatting (f-strings)

```
name = "Aman"
age = 25
print(f"My name is {name}
      and age is {age}")
```

Strings are immutable, i.e., cannot be changed.

Python Cheat Sheet

8. Lists (Mutable)

- Declaration

```
lst = [1, 2, 3]
```

- Indexing / Slicing

```
lst[0]    # 1
lst[1:3]  # [2, 3]
```

- Update / Change

```
lst[1] = 10
```

- Methods
  - append(x)
  - extend(iterable)
  - insert(i, x)
  - remove(x)
  - pop([i])
  - clear()
  - index(x)
  - count(x)
  - sort()
  - reverse()

Example

```
lst = [1, 2, 3]
lst.append(4)
lst → [1, 2, 3, 4]
```

9. Tuples (Immutable)

- Declaration

```
t = (1, 2, 3)
```

- Indexing / Slicing

```
t[0]    # 1
t[1:3]  # (2, 3)
```

- Methods
  - count(x)
  - index(x)

Tuples are immutable i.e., cannot be changed after creation.

10. Sets (Unordered, Unique)

- Declaration

```
s = {1, 2, 3}
```

- Add / Remove
  - add(x)
  - update(iterable)
  - remove(x)
  - discard(x)
  - pop()
  - clear()
- Set Operations
  - union (s1 | s2)
  - intersection (s1 & s2)
  - difference (s1 - s2)
  - symmetric diff (s1 ^ s2)

11. Dictionary (Key-Value)

- Declaration

```
d = {"a": 1, "b": 2}
```

- Access / Update

```
d["a"]    # 1
d["c"] = 3
```

- Methods
  - keys()
  - values()
  - items()
  - get(key, default)
  - pop(key, default)
  - popitem()
  - update(other_dict)
  - clear()
  - fromkeys(seq, value)

12. Loops

For Loop

```
for i in range(5):
    print(i)

for item in iterable:
    print(item)
```

While Loop

```
while condition:
    statements
```

```
break    → exit loop
continue → next iteration
pass     → do nothing
```

13. Conditionals

if / elif / else

```
if condition:
    statements
elif another_condition:
    statements
else:
    statements
```

Truthy Values

True, non-empty sequence, non-zero numbers, etc.

Falsy Values

False, None, 0, "", [], (), {}

14. Comprehensions

List Comprehension

```
squares = [x*x for x in range(5)]
```

Dict Comprehension

```
d = {x: x*x for x in range(5)}
```

Set Comprehension

```
s = {x for x in range(5) if x%2==0}
```

15. Functions

- Define Function

```
def add(a, b):
    return a + b
```

- Default Arguments

```
def greet(name, msg="Hi"):
    return msg + ", " + name
```

- *args and **kwargs

```
def fun(*args, **kwargs):
    pass
```

Example

```
add(2, 3)          # 5
greet("Aman")      # Hi, Aman
fun(1, 2, 3, a=10, b=20)
```

16. Lambda, Map, Filter, Reduce

Lambda (anonymous function)

```
square = lambda x: x*x
print(square(5))   # 25
```

Map

```
nums = [1, 2, 3]
sq = list(map(lambda x: x*x, nums))
```

Filter

```
evens = list(filter(lambda x: x%2==0, nums))
```

Reduce (from functools)

```
from functools import reduce
from operator import add
total = reduce(add, nums)   # 6
```

17. Modules & Packages

- Import Module

```
import math
from math import sqrt
```

- Install Package

```
pip install package_name
```

- Create Module

```
# mymodule.py
def hello():
    print("Hello")
```

Example

```
import math
print(math.pi)
print(sqrt(16))
```

A module is a .py file
A package is a directory with __init__.py file.